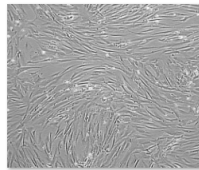


10 Laboratory – Bayes Classifier

1 Introduction

A given microscopic image of muscle cells should be categorized as representing one of the four types of muscle cells shown in the images below. For this you will implement a Bayes classifier. From the microscopic image, four features will be extracted from its co-occurrence matrix to form a feature vector $\mathbf{x} \in \mathbb{R}^4$. **Note** that in this lab, the number of features in a feature vector is four and the number of classes is also four by coincidence. In general however, they will be different.



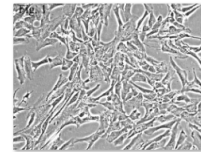
mc1.tif



mc2.tif



mc3.tif



mc4.tif

Bayes Classifier. Simplifying assumptions for the classifier are:

- The likelihoods $p(\mathbf{x}|K_j)$ for $j \in \{1, 2, 3, 4\}$ are assumed to be multivariate Gaussian, i.e. expressed completely by mean vectors $\mathbf{m}_j$ and covariance matrices $\mathbf{C}_j$.
- The loss function $L_{ij} = 1$ if $i \neq j$ and 0 otherwise.
- The prior probabilities $p(K_j)$ are identical for all classes.

Under these assumptions, the Bayes classification rule simplifies to

$$\hat{j} = \arg \max_j \left\{ d_j(\mathbf{x}) \right\},$$

with

$$d_j(\mathbf{x}) = \ln(P(K_j)) - \frac{1}{2} \ln(|\mathbf{C}_j|) - \frac{1}{2} (\mathbf{x} - \mathbf{m}_j)^T \mathbf{C}_j^{-1} (\mathbf{x} - \mathbf{m}_j),$$

from which we can skip the constant term $\ln(P(K_j))$ as $K_j = K_i$ for all i, j , i.e. we can use

$$d_j(\mathbf{x}) = -\ln(|\mathbf{C}_j|) - (\mathbf{x} - \mathbf{m}_j)^T \mathbf{C}_j^{-1} (\mathbf{x} - \mathbf{m}_j).$$

Feature Generation. For this lab, 96 images were analyzed. Specifically, the co-occurrence matrix for the grayscale image was calculated and based on this, the *energy*, *contrast*, *entropy*, *homogeneity* measures were calculated. The 96 training vectors were prepared for your ease. They are divided into a training set and a test set.

Training Set. The training feature vectors are available for use with matlab and python in the files `featuresForTraining.mat` and `train_data.pkl`, respectively. The training vectors contain an equal number of representatives of each of the classes 1 to 4.

Test Set. The Feature vectors for testing have also been computed from the images below and made available in `featuresForTesting.mat` and `test_data.pkl`.

The test data contains an equal number of representatives of each of the classes 1 to 4.

2 Tasks

- a) The reading of the training and test data is prepared. Make yourself known with the code e.g. by inspecting variables in debug mode.
- b) Complete the function `train` by determining the class centers $\mathbf{m}_j \in \mathbb{R}^4$ and the covariance matrices $\mathbf{C}_j \in \mathbb{R}^{4 \times 4}$ for all four classes. The mean value $\mathbf{m}_j$ for class j is formed by the N training vectors $\mathbf{x}_{jn}$ for $n \in 1, 2, \dots, N$:

$$\mathbf{m}_j = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_{jn}$$

The covariance is formed by

$$\begin{aligned} \mathbf{C}_j &= \frac{1}{N-1} \sum_{n=1}^N (\mathbf{x}_{jn} - \mathbf{m}_j) \cdot (\mathbf{x}_{jn} - \mathbf{m}_j)^T \\ &= \frac{1}{N-1} \mathbf{D}_j \mathbf{D}_j^T, \end{aligned}$$

where $\mathbf{D}_j = [\mathbf{d}_{j1}, \mathbf{d}_{j2}, \dots, \mathbf{d}_{jN}]$, with $\mathbf{d}_{jn} := \mathbf{x}_{jn} - \mathbf{m}_j$.

- c) Implement the test functions `train`, `classify`, and optionally `computeConfusionMatrix`.
- d) Run the function `main`. You should not observe any misclassifications, as the training and test data sets have been tuned to just achieve perfect classification.
- e) Now degrade the classifier by assuming that the covariance matrix is the unity matrix. As a result you should observe misclassifications.